

BRAC University
Department of Computer Science and Engineering

Examination: Quiz 03

Duration: 30 Minutes

Semester: Summer 26

Full Marks: 15

CSE420: Compiler Design

1. Consider the following SDT. Draw the Annotated Parse Tree and Evaluate the values of the attributes type and width along with variable offset next to the appropriate non-terminal nodes and populate the symbol table for the given String.

Input String:

```
int a;
  record {
    float [4]b;
  } recor;
```

SDT:

P → **S D**

S → {*offset* = 400; *table* = new *SymTable*(); *o* = new *Stack*(); *t* = new *Stack*(); } **F**

D → **T id** ; {*sym* = new *Symbol*(); *sym.type* = *T.type*; *sym.width* = *T.width*;
 table.insert(*id.lexeme*, *sym*); *offset* = *offset* + *T.width*; } **D₁**

D → ε

F → ε

T → **B** {*C.bT* = *B.t*; *C.bW* = *B.w*; } **C** {*T.type* = *C.type*; *T.width* = *C.width*}

T → **record** {*o.push*(*offset*); *t.push*(*table*); *offset* = *offset* + 100 - 1; *table* = new *SymbolTable*(); }
 ‘{ **D** ’ {*T.type* = *table*; *T.width* = *offset*; *offset* = *o.pop*(); *table* = *t.pop*(); }

B → **int** { *B.t* = *int*; *B.w* = 5; }

B → **float** { *B.t* = *float*; *B.w* = 6; }

C → ε {*C.type* = *C.bT*; *C.width* = *C.bW*; }

C → [**num**] {*C₁.bT* = *C.bT*; *C₁.bW* = *C.bW*; }
 C₁ {*C.type* = *array*(*num.value*, *C₁.type*); *C.width* = *C₁.width* * (*num.value*); }

Name:	ID:	Section:
-------	-----	----------

BRAC University
Department of Computer Science and Engineering

Examination: Quiz 03

Duration: 30 Minutes

Semester: Summer 26

Full Marks: 15

CSE420: Compiler Design

1. Consider the following SDT. Draw the Annotated Parse Tree and Evaluate the values of the attributes type and width along with variable offset next to the appropriate non-terminal nodes and populate the symbol table for the given String.

Input String:

```
int [4]b;
  record {
    float c;
  } recor;
```

SDT:

P → {offset = 0; table = new SymTable(); o = new Stack(); t = new Stack(); } **D**

D → **T id** ; {sym = new Symbol(); sym.type = T.type; sym.width = T.width;
table.insert(id.lexeme, sym); offset = offset + T.width; } **D₁**

D → ε

T → **B** {C.bT = B.t; C.bW = B.w;} **C** {T.type = C.type; T.width = C.width}

T → **record** {o.push(offset); t.push(table); offset = offset + 100 - 1; table = new SymbolTable(); }
'**D** '**C**' {T.type = table; T.width = 1 * offset; offset = o.pop(); table = t.pop();}

B → **int** { B.t = int; B.w = 5; }

B → **float** { B.t = float; B.w = 6; }

C → ε {C.type = C.bT; C.width = C.bW; }

C → [**num**] {C₁.bT = C.bT; C₁.bW = C.bW;}
C₁ {C.type = array(num.value, C₁.type); C.width = C₁.width * (num.value);}

Name:	ID:	Section:
-------	-----	----------

BRAC University
Department of Computer Science and Engineering

Examination: Quiz 03

Duration: 30 Minutes

Semester: Summer 26

Full Marks: 15

CSE420: Compiler Design

1. Consider the following SDT. Draw the Annotated Parse Tree and Evaluate the values of the attributes type and width along with variable offset next to the appropriate non-terminal nodes and populate the symbol table for the given String.

Input String:

```
record {  
    float c;  
} rer;  
int [4] b;
```

SDT:

P → **G D**

G → {*offset* = 800; *table* = new *SymTable*(); } **F**

F → {*o* = new *Stack*(); *t* = new *Stack*();} **E**

E → ε

D → **T id** ; {*sym* = new *Symbol*(); *sym.type* = *T.type*; *sym.width* = *T.width*;
 table.insert(*id.lexeme*, *sym*); *offset* = *offset* + *T.width*; } **D₁**

D → ε

T → **B** {*C.bT* = *B.t*; *C.bW* = *B.w*; } **C** {*T.type* = *C.type*; *T.width* = *C.width*}

T → **record** {*o.push*(*offset*); *t.push*(*table*); *offset* = *offset* + 100 - 1; *table* = new *SymTable*(); }
 ‘{ **D** ’ } {*T.type* = *table*; *T.width* = *offset* - 100; *offset* = *o.pop*(); *table* = *t.pop*();}

B → **int** { *B.t* = *int*; *B.w* = 5; }

B → **float** { *B.t* = *float*; *B.w* = 6; }

C → ε {*C.type* = *C.bT*; *C.width* = *C.bW*; }

C → [**num**] {*C₁.bT* = *C.bT*; *C₁.bW* = *C.bW*;}
 C₁ {*C.type* = *array*(*num.value*, *C₁.type*); *C.width* = *C₁.width* * (*num.value*);}

Name:	ID:	Section:
-------	-----	----------

BRAC University
Department of Computer Science and Engineering

Examination: Quiz 03
Duration: 30 Minutes

Semester: Summer 26
Full Marks: 15

CSE420: Compiler Design

1. Consider the following SDT. Draw the Annotated Parse Tree and Evaluate the values of the attributes type and width along with variable offset next to the appropriate non-terminal nodes and populate the symbol table for the given String.

Input String:

```
float [2] [3]c;  
record {  
  
    } r;
```

SDT:

P → **S D**

S → { *offset = 100; table = new SymTable(); o = new Stack(); t = new Stack();* } **F**

D → **T id** ; { *sym = new Symbol(); sym.type = T.type; sym.width = T.width;*
table.insert(id.lexeme, sym); offset = offset + T.width; } **D₁**

D → ε

F → ε

T → **B** { *C.bT = B.t; C.bW = B.w;* } **C** { *T.type = C.type; T.width = C.width* }

T → **record** { *o.push(offset); t.push(table); offset = offset + 100 - 1; table = new SymbolTable();* }
'{ ' **D** '}' { *T.type = table; T.width = 100; offset = o.pop(); table = t.pop();* }

B → **int** { *B.t = int; B.w = 5;* }

B → **float** { *B.t = float; B.w = 6;* }

C → ε { *C.type = C.bT; C.width = C.bW;* }

C → [**num**] { *C₁.bT = C.bT; C₁.bW = C.bW;* }
C₁ { *C.type = array(num.value, C₁.type); C.width = C₁.width * (num.value);* }

Name:	ID:	Section:
-------	-----	----------